Listen Labs

ROI Calculator — Product Technical Specification

Version 1.0 · For engineering handoff

Document type	Product & Engineering Spec
Product	Listen Labs ROI Calculator — interactive web tool
Purpose	PLG conversion asset — helps marketing buyers justify Listen to their CFO
Target audience	Frontend SWE (React/TypeScript). Design handoff included.
Live prototype	Interactive demo available on request (Claude.ai artifact)

1. Product Overview

The Listen Labs ROI Calculator is a multi-screen interactive web experience that turns a marketing manager's research question into a visceral "before and after" story — showing exactly what slow, expensive traditional research is costing them, then dramatically revealing how Listen compresses that cost and timeline.

Primary goal: convert marketing buyers who need to justify Listen to their CFO. The calculator does the CFO-justification work for them.

1.1 User journey

The experience flows across four sequential screens:

Screen	Name	What happens	Duration

1	Input	User types research question + sets participant count via slider. Voice input supported. Live tier preview updates in real time.	~15 sec
2	Processing	Gear animation plays. Status messages update showing participant-specific analysis. Progress bar fills across 4 steps.	~3 sec
3	Traditional timeline	Gantt chart builds phase by phase. Each bar animates in left-to-right. Donut chart slice fills in sync. Budget counter ticks up. Legend populates. Totals reveal.	~8 sec
4	Listen reveal	RAG search animation runs (company name shuffle). Match found. Listen Gantt appears. Donut animates freed budget. Case study card reveals. Savings callout appears.	~6 sec

2. Inputs & Data Model

2.1 User inputs

Input	Type	Default / Range	Behaviour
Research question	Textarea	Placeholder cycles every 3.5s	Auto-grows. ⌘↵ triggers analysis. Cycling placeholder: 3 example prompts. Voice input via Web Speech API.
Participant count	Range slider	Default 20, range 5–100	On input: updates nout display, tier label, and live preview line showing traditional vs Listen estimates. Triggers recalculation on submit.

2.2 Tier system

All cost, time, and animation data is derived from participant count at calculation time. Five tiers are defined:

Participants	Label	Trad. weeks	Trad. cost	Listen time	Listen note
5–8	Small concept test	4 wks	$12,000	4–6 hours	All interviews done in one session
9–15	Medium qual study	6 wks	$18,000	6–12 hours	Participants respond within a few hours
16–25	Standard IDI study	8 wks	$25,000	Same day	Most responses arrive within 12–24 hrs
26–50	Large research program	12 wks	$40,000	1–2 days	Panel fully closes within 48 hours
51–100	Enterprise study	16 wks	$60,000	2–3 days	Staggered global recruitment closes by day 3

Listen cost per tier: $1,200 / $1,800 / $2,500 / $4,000 / $8,000. Freed budget = traditional cost minus Listen cost. "More studies" = floor(freed / Listen cost).

2.3 Traditional phase cost model

Phase costs are derived from the tier total using fixed proportions (cf = cost fraction). These fractions must sum to 1.0.

Phase name	Row	cf	@ $25K	Color	Notes
Screener & guide	Top	0.08	$2,000	#378ADD	fs: 0.000, fd: 0.125
Recruit participants	Bot	0.20	$5,000	#D85A30	fs: 0.125, fd: 0.250
Schedule interviews	Top	0.04	$1,000	#EF9F27	fs: 0.375, fd: 0.100
In-depth interviews	Bot	0.32	$8,000	#7F77DD	fs: 0.475, fd: 0.188
Transcription	Top	0.12	$3,000	#1D9E75	fs: 0.663, fd: 0.113
Analysis & synthesis	Bot	0.16	$4,000	#D4537E	fs: 0.775, fd: 0.113
Report & deck	Top	0.08	$2,000	#639922	fs: 0.888, fd: 0.113

Cost per phase = round(cf × tier.total / 500) × 500 (rounded to nearest $500)

Bar start position in SVG = LO + (fs × tier.weeks × WP) where LO=15, WP=(431−15)/tier.weeks

Bar width in SVG = fd × tier.weeks × WP

3. Screen-by-Screen Specification

3.1 Screen 1 — Input

Layout

- Full-width textarea (min-height 96px, auto-grows on input)
- Participant count card below textarea with: range slider (5–100), live count display, 3 anchor labels (5/25/100), live tier preview line
- Bottom row: voice button (left), ⌘↵ hint, calculate CTA (right)

Textarea behaviour

- Placeholder cycles every 3,500ms through 3 examples when empty
- Auto-height: on every input event, set height=auto then height=scrollHeight
- ⌘↵ (Mac) and Ctrl+↵ (Windows) submit the form

Tier preview line

Updates live on every slider input event. Format:

```
  [getTierLabel(n)]  ·  traditionally ~[t.weeks] weeks / $[t.total]  ·  with Listen [t.ltime]
```

Traditional figures are styled in danger color (red), Listen figure in success color (green).

Voice input

- Detect window.SpeechRecognition || window.webkitSpeechRecognition
- On click: start() recognition, set interimResults=true, show "listening..." status, change button text color to danger
- onresult: join all transcript items, write to textarea value, dispatch input event to trigger auto-grow
- onend: reset button state and status text
- If API unavailable: disable button, opacity 0.35

3.2 Screen 2 — Processing

Elements

- Prompt preview card — grey background, shows ta.value verbatim
- Two interlocking SVG gears, CSS keyframe rotation
- Status text — cycles through 4 messages with participant count interpolated
- Progress bar — fills 25% per message (100ms transition)

Gear specification

Gears are generated programmatically from a path-drawing function. Two gears mesh visually:

- Gear A: cx=32, cy=32, outer R=28, inner R=20, N=10 teeth. Rotation: 3s linear infinite clockwise. transform-origin: 32px 32px
- Gear B: cx=71, cy=55, outer R=18, inner R=13, N=7 teeth. Rotation: 2.1s linear infinite counter-clockwise. transform-origin: 71px 55px

The rotation speeds (3s vs 2.1s) are set so that tooth-counts are inversely proportional: 10/7 × 3 ≈ 2.1, producing a realistic meshing illusion.

Status messages (participant-aware)

- "identifying research type..."
- "estimating panel recruitment time ([n] participants)..."
- "calculating [n]-person interview schedule..."
- "adding transcription & analysis costs..."

Each message shows for 700ms. After all 4 messages, transition to Screen 3 after a 500ms pause.

3.3 Screen 3 — Traditional Timeline

Layout

- Section header: "the traditional approach" (uppercase, small) + right-aligned "[n] participants · [tierLabel]" tag
- Main row: Gantt SVG (flex:1) + Donut SVG (152×152px fixed width)
- Legend strip: flex-wrap row of color-dot + phase name + cost badges
- Stats footer: "total time" (left) + "total cost" (right), large type
- "There's another way" CTA — appears after all phases + final budget animation complete

Gantt SVG

ViewBox: 0 0 440 178. preserveAspectRatio xMidYMid meet. Width 100%. Responsive.

Axis: Horizontal line at y=86, x=15 to x=431

Week ticks: Vertical ticks at y=81–91 for each week. Labels at y=105. Show every ceil(weeks/8)th tick or all ticks if weeks≤8

Phase bars (top): y=38, height=38, border-radius=3, fill with 85% opacity

Phase bars (bottom): y=95, height=38, border-radius=3, fill with 85% opacity

Phase name labels: Top bars: y=30 above bar. Bottom bars: y=150 below bar. Font-size 10, fill=phase color, font-weight 500

Cost labels: Inside bar, y=barY+barH−9, font-size 9, fill white, opacity 0.9

Connector line: Dashed (2,2) vertical line from bar edge to label, opacity 0.6

Gantt animation sequence

Phases appear one at a time with an 800ms interval between each. Per phase:

1. Append bar (width=0), cost label (opacity=0), name label (opacity=0), connector (opacity=0)
2. requestAnimationFrame: begin animW() — ease-in-out width expansion over 520ms
3. After 180ms: fade in labels + connector
4. After 200ms: begin animN() — budget counter eases from prev to prev+cost over 380ms
5. Simultaneously: reveal corresponding donut slice (stroke-dasharray transition)
6. Fade in corresponding legend item

After all 7 phases: update total time label → run final animN to exact tier.total → show CTA after 400ms.

Donut chart

SVG 152×152, viewBox 0 0 152 152. Center 76,76. Ring: r=56, stroke-width=22.

Background ring: stroke=var(--color-background-secondary). Hole: filled circle r=45, fill=var(--color-background-primary).

Each phase slice: <circle> with matching stroke color, computed dashoffset and dasharray:

```
circumference C = 2π × 56 ≈ 351.86
slice.dashoffset = C/4 − (sum_of_previous_fracs × C)
slice.dasharray (initial) = "0 C"
slice.dasharray (revealed) = "[frac×C] [C−frac×C]"
CSS transition: stroke-dasharray 0.6s cubic-bezier(.4,0,.2,1)
```

Center overlay: absolute-positioned div, "total cost" label + budget number. Number bumps (scale 1→1.09→1) on each update.

3.4 Screen 4 — Listen Reveal

Phase A: RAG search animation (~2.6 seconds)

Two side-by-side info cards inside a container div:

- "Scanning" card — cycles through 7 status lines at 300ms intervals, each line updating the progress bar
- "Comparing studies" card — shuffles through 12 company+use-case strings at 90ms intervals

Status lines are participant-count-aware, e.g. "Filtering by study size (20 participants)..."

At 2,600ms: clearInterval on both timers. Set scanning text to "Match found." Set shuffle text to matched company. Progress bar to 100%. Show match badge with fadeUp animation.

At 3,400ms: fade out RAG div (opacity 0, transition 0.4s). At 3,800ms: hide RAG, call buildResult().

Match logic

Keyword regex matching on lowercased userPrompt. First match wins, falls back to Microsoft:

Match	Pattern	Case study
Microsoft	/onboard|activation|launch|copilot|enterprise|feature/	"What once took 6–8 weeks now happens in days."
Sweetgreen	/price|menu|concept|brand|perception|consumer/	"Understanding outliers helps unlock our best ideas."

Canva	/ux|usability|design|product|feature|journey/	Continuous qual insights across product & growth.
Sling Money	/churn|cancel|retain|drop|friction|payment/	"Create a survey in 10 minutes, results same day."

Phase B: Listen result

Layout mirrors Screen 3: Listen Gantt SVG (flex:1) + Listen donut (152px fixed).

Listen Gantt

ViewBox: 0 0 440 88. Axis at y=57. 4 bars at y=15, height=33. Dynamic time axis based on tier.axisTicks.

Phase	start%	end%	Color	Time sublabel (dynamic)
Guide	0%	5%	#9FE1CB	fmtH(0) – fmtH(0.05 × axisH)
Recruit	5%	15%	#5DCAA5	fmtH(0.05 × axisH) – fmtH(0.15 × axisH)
Interviews	15%	65%	#1D9E75	fmtH(0.15 × axisH) – fmtH(0.65 × axisH)
Analysis	65%	100%	#0F6E56	fmtH(0.65 × axisH) – fmtH(axisH)

fmtH(h): if h===0 → "now"; if h<1 → round(h×60)+"min"; if integer → h+"hr"; else h.toFixed(1)+"hr"

Axis ticks placed evenly across x=20 to x=420, spaced by 400/(nTicks−1). Labels from tier.axisTicks array.

Bars animate in sequentially at 360ms intervals (animW). Label + sublabel fade in 250ms after bar starts.

Listen donut — freed budget

Two new circles inserted before the hole circle:

- Listen cost slice: stroke #1D9E75, dashoffset=C/4. Animates to [LF×C, C×(1−LF)]. Transition 0.7s.
- Freed budget slice: stroke #B5D4F4 (light blue), dashoffset=C/4−LF×C. Animates to [SF×C, C×(1−SF)]. Transition 1.3s, begins 600ms after cost slice.

Center overlay: "$0" → animN to (tier.total − tier.lcost) over 1,400ms. On complete → reveal case study card.

Reveal sequence timing

Delay	Event
0ms	Listen Gantt bars begin animating (bar 1 at 100ms, bar 2 at 460ms, bar 3 at 820ms, bar 4 at 1180ms)
~1700ms	Listen cost donut slice animates in
~2300ms	Freed budget donut slice animates in. Budget counter begins.
~3700ms	Budget counter reaches final value. Case study card fadeUp.
~4200ms	Savings callout card fadeUp.

4. Animation System

4.1 Core easing function

All number and width animations use a single ease-in-out quadratic function:

```
function easeInOut(p) {
  return p < 0.5 ? 2*p*p : -1 + (4 - 2*p) * p;
}
```

4.2 animN — number animation

```
animN(from, to, duration, onUpdate, onComplete)
```

Uses requestAnimationFrame. Calculates progress p = min((now−t0)/dur, 1). Calls onUpdate with Math.round(from + (to−from) × ease(p)).

4.3 animW — SVG width animation

```
animW(svgRectElement, targetWidth, duration=520)
```

Same RAF loop as animN, calls el.setAttribute("width", value.toFixed(1)) on each frame.

4.4 CSS animations

Name	Trigger	Spec
rot	Gear A	3s linear infinite. from 0deg to 360deg.
rot2	Gear B	2.1s linear infinite. from 0deg to −360deg.
fu	Cards, CTAs	from {opacity:0; translateY(10px)} to {opacity:1; translateY(0)}. 0.3–0.5s ease forwards.

bmp	Budget number	0.22s ease. scale: 1 → 1.09 → 1. Applied via class add/ remove with reflow flush.

5. Technical Stack & Requirements

5.1 Recommended stack

- React 18 + TypeScript
- Vite for bundling
- No charting library needed — all visuals are hand-drawn SVG
- No CSS framework required — use CSS custom properties / design tokens
- Framer Motion optional for page transitions (the RAF-based animations are custom)

5.2 Browser requirements

- Chrome / Edge 90+, Safari 15+, Firefox 90+
- Voice input: Chrome/Edge only (webkitSpeechRecognition). Graceful degradation on unsupported browsers.
- SVG CSS transitions on stroke-dasharray: all modern browsers ✓
- Must be mobile-responsive. Gantt SVG uses preserveAspectRatio="xMidYMid meet" + width:100% to scale.

5.3 State management

All state is local component state — no global store required. Key state variables:

- userPrompt: string — the research question
- nParticipants: number (5–100) — from slider
- currentTier: Tier object — computed from nParticipants on submit
- currentScreen: "input" | "processing" | "traditional" | "reveal"
- curBudget: number — running tally for animation
- phaseIndex: number — which traditional phase is currently animating

5.4 Data structures

Define these as typed constants (TypeScript interfaces provided):

```
interface Tier {
  max: number;       // upper bound of participant range
  weeks: number;     // traditional study duration in weeks
  total: number;     // traditional total cost
  ltime: string;     // Listen time estimate (display string)
  lnote: string;     // explanatory footnote for Listen time
  lcost: number;     // Listen cost
  axisH: number;     // hours for Listen Gantt axis
  axisTicks: string[];// labels for Listen Gantt time axis
}

interface Phase {
  sh: string;        // short display name
  fs: number;        // fractional start position (0–1 of total weeks)
  fd: number;        // fractional duration (0–1 of total weeks)
  row: "t" | "b";    // top or bottom row in Gantt
  cf: number;        // cost fraction (sums to 1.0 across all phases)
  c: string;         // hex color
  // Computed at runtime:
  sw: number;        // start week = fs × tier.weeks
  dw: number;        // duration weeks = fd × tier.weeks
  cost: number;      // = round(cf × tier.total / 500) × 500
}
```

```
interface CaseStudy {
  co: string;        // company name
  tag: string;       // industry label
  use: string;       // use case description
  trad: string;      // traditional approach (display string)
  q: string;         // quote
  who: string;       // attribution
  keys: RegExp;      // keyword matching regex
}
```

6. Shareable Link Spec

The production version should be sharable with pre-filled state via URL query params. This allows sales reps to pre-fill the calculator for specific prospects.

Param	Type	Behaviour
?q=	string (URL encoded)	Pre-fills the research question textarea. Auto-triggers calculation on load if present.
?n=	integer 5–100	Pre-sets participant slider. Defaults to 20 if absent or out of range.
?s=	"traditional"|"reveal"	Skip to that screen on load (for sharing a result). Optional.

Example: /roi?q=Why+are+users+churning+in+month+3%3F&n=20&s=reveal

Priority: implement ?q and ?n first. The auto-calculate-on-load behaviour is the highest-value feature for the sales use case.

7. Design Tokens & Visual Language

7.1 Color usage

Token	Use	Notes
--color-text-danger	Traditional costs, budget counter	Red. Signals pain.
--color-text-success	Listen time, freed budget, CTAs	Green. Signals relief.
--color-text-info	Processing screen, CTA buttons	Blue. Signals action.
--color-background-secondary	Prompt preview card, tier card, RAG cards	Subtle surface.
--color-background-success	Savings callout card	Green wash.
Phase colors (7)	Gantt bars + donut slices (must match)	See phase table § 2.3.
#B5D4F4	Freed budget donut arc	Light blue — "potential."

7.2 Typography

- Body: var(--font-sans), 14–15px, weight 400, line-height 1.7
- Section labels: 11px, weight 500, letter-spacing 0.07em, text-transform uppercase
- Hero numbers (totals): 24–26px, weight 500
- Code/monospace: var(--font-mono) for RAG scan lines only

7.3 Spacing & radii

- Cards: padding 12–16px, border-radius var(--border-radius-lg) (12px)
- Buttons: padding 8–9px vertical / 14–22px horizontal, border-radius var(--border-radius-md) (8px)
- SVG bars: rx=3 (subtle rounding)
- Borders: 0.5px lines for dividers, 1.5px for section rules, 3px left-accent for case study card

8. Acceptance Criteria

8.1 Functional

- Participant slider from 5 to 100 updates tier preview in real time without lag
- Traditional Gantt phases appear sequentially — each bar must finish width animation before next phase starts OR use the staggered timeout model in spec § 3.3
- Donut slice for each phase fills in sync with the corresponding Gantt bar appearance
- Budget counter updates smoothly — no integer jumps, no negative deltas
- RAG shuffle text updates at ~90ms. Status lines update at ~300ms. Neither freezes the UI.
- Case study keyword matching works for all 4 patterns. Falls back to Microsoft.
- Voice input works in Chrome. Button is disabled (not hidden) in unsupported browsers.
- "Try another study size" resets to Screen 1 with the slider and textarea preserved

8.2 Visual

- Phase color on Gantt bar matches corresponding donut slice (same hex, no drift)
- Listen Gantt time axis labels match the tier's axisTicks array exactly
- Listen time estimate in header matches tier.ltime exactly (not hardcoded "4 hours")
- Freed budget donut animates from 0 to saved value — never shows final value before animation
- No label overlap on either Gantt at any screen width ≥ 320px
- All cards fade in with the fu animation — no content pops in without transition

8.3 Performance

- Time to interactive < 2s on a mid-range mobile device
- All SVG animations maintain 60fps — no jank during concurrent Gantt + donut + counter animations
- No external runtime dependencies beyond React + optional Framer Motion

9. Out of Scope (V1)

- Backend API or database — all data is hardcoded in the client
- Auth or user accounts
- Saving or persisting results
- Email capture or CRM integration (V2)
- Analytics tracking (add PostHog or Mixpanel in V2)
- A/B testing framework
- Custom branding / white-labeling

10. Open Questions for Engineering

- Hosting: standalone Vercel deploy vs embedded iframe on listenlabs.ai?
- Should the shareable URL include a "book demo" CTA that links to the live Calendly? Confirm URL.
- Are the Listen cost tiers ($1,200 / $1,800 / $2,500 / $4,000 / $8,000) accurate to actual Listen pricing? Needs confirmation from Listen team before launch.
- Do we want a "Download as PDF" button on the results screen for the CFO use case?
- Mobile: should the Gantt + Donut layout stack vertically on viewports < 480px?

End of spec. Questions? Ping the author.